Turtle Creative Coding — Technical Function Reference

Code-based reference | 22 October 2026 | Based on the available updated turtle_shapes_speed_fixed.py and teacher_sequence_speed_fixed.py

Scope: documents the supplied source files, not a verified GitHub release. Rename the fixed files to turtle_shapes.py and teacher_sequence.py when installing. The student_images.py file and other dependencies were not available for this audit.

1. Quick start

```
from turtle_shapes import *

stars = []
for i in range(5):
    stars += star(x=-4 + i * 2, y=0, size=1, color="purple", speed=20)

# Add ("Stars", stars) to your existing image_sequence.
```

Shape functions return lists of instruction tuples. They do not draw immediately. The student application passes these lists to the teacher engine.

2. Shape and pattern functions

triangle()

```
triangle(x=0, y=0, size=3, angle=0, color='blue', fill=None, speed=None, width=3)
```

Equilateral triangle; 3 sides, 120° turns.

square()

```
square(x=0, y=0, size=2, angle=0, color='green', fill=None, speed=None, width=3)
```

Square; delegates to rectangle().

rectangle()

```
rectangle(x=0, y=0, width=4, height=2, angle=0, color='blue', fill=None, speed=None, line_width=3)
```

Rectangle; width and height specify side lengths, while line_width specifies pen thickness.

Important: rectangle(width=...) changes the geometric width; rectangle(line_width=...) changes the pen thickness. Other shape functions use width for pen thickness.

pentagon()

pentagon(x=0, y=0, size=2, angle=0, color='orange', fill=None, speed=None, width=3)

Regular 5-sided polygon.

hexagon()

hexagon(x=0, y=0, size=2, angle=0, color='purple', fill=None, speed=None, width=3)

Regular 6-sided polygon.

star()

star(x=0, y=0, size=3, angle=0, color='gold', fill=None, speed=None, width=3)

Five-point star; 144° turns.

```
one_star = star(x=-2, y=1, size=2, color="purple", width=5, speed=20)
```

circle()

circle(x=0, y=0, radius=1, angle=0, color='purple', fill=None, speed=None, width=3)

Full circular curve (360°).

The radius is in grid units. The current implementation creates a full circle using the curve instruction.

spiral()

spiral(x=0, y=0, turn=90, steps=14, growth=0.25, angle=0, color='purple', speed=None, width=3)

Increasing line segments separated by turns.

letter_a()

letter_a(x=0, y=0, size=3, color='red', speed=None, width=4)

Draws an A from strokes; fixed internal angles.

letter_h()

letter_h(x=0, y=0, size=3, color='purple', speed=None, width=4)

Draws an H from strokes; fixed internal angles.

letter_s()

letter_s(x=0, y=0, radius=1, color='purple', speed=None, width=4)

Two connected, opposite 180° arcs; no angle parameter.

The S uses radius for both arcs and width for line thickness. To alter arc angles, use raw ("curve", radius, degrees) commands or modify this function.

```
my_s = letter_s(x=0, y=2, radius=1, color="purple", speed=5, width=8)
```

venetian_blind()

venetian_blind(x=-4, y=-3, width=8, slats=7, slat_height=0.55, gap=0.3, color='steelblue', fill='lightblue', speed=None, line_width=2)

Repeated horizontal filled rectangles (slats).

The y coordinate is the bottom of the first slat; subsequent slats are spaced by slat_height + gap.

zigzag()

zigzag(x=-4, y=0, steps=6, segment=1, angle=45, color='orange', speed=None, width=3)

Alternating turns to make a zigzag.

repeat_shape()

repeat_shape(shape_function, count=3, x=0, y=0, dx=1, dy=0, **kwargs)

Repeats a chosen shape function at regularly offset positions.

The shape_function argument must be a callable shape function, such as star or triangle, not its returned instruction list.

growing_shapes()

growing_shapes(shape_function, count=3, x=0, y=0, start_size=1, step=0.5, **kwargs)

Repeats a shape at one location with increasing size.

The shape_function argument must be a callable shape function, such as star or triangle, not its returned instruction list.

random_colors()

random_colors(shape_function, count=4, palette=None, x=0, y=0, dx=1, dy=0, **kwargs)

Repeats a shape with randomly selected palette colours.

The shape_function argument must be a callable shape function, such as star or triangle, not its returned instruction list.

3. Common parameters

Parameter	Meaning / caution
x, y	Starting coordinates in grid units; classroom grid shows approximately −5 to +5.
size	Side or stroke length, generally in grid units.
radius	Radius of circular arcs, in grid units.
angle	Starting heading in degrees for shapes that accept it; NOT a letter_s() parameter.
color	Pen/outline colour, e.g. "purple".
fill	Interior colour; None disables filling. True uses the outline colour.
speed	Optional per-shape animation speed; None

	preserves current engine speed.
width	Pen thickness for most shapes; geometric width for rectangle() and venetian_blind().
line_width	Pen thickness for rectangle() and venetian_blind().
count, dx, dy	Number of copies and positional offsets for repeat helpers.
palette	List of colours for random_colors().

4. Drawing instruction tuples

Command	Effect
("up", x, y)	Lift pen and travel to position.
("up",)	Lift pen without moving.
("down",)	Lower pen.
("heading", degrees)	Set absolute direction.
("turn", degrees)	Turn left relative to current heading.
("forward", distance)	Move forward in grid units.
("curve", radius, degrees)	Draw arc; negative degrees reverse turn direction.
("color", name)	Change pen colour.
("fillcolor", name)	Change fill colour.
("width", number)	Change pen thickness; must be > 0.
("begin_fill",)	Begin filled shape.
("end_fill",)	Finish filled shape.
("speed", integer)	Set drawing speed; fixed engine supports 0–100.
("repeat", n) ... ("end_repeat",)	Repeat block n times; nested repeats supported.
("repeat_grow", n) ... ("end_repeat",)	Expand repeated block with growing size placeholder.
("random_fill", palette)	Choose a random fill colour.
("sound", name)	Play named sound.
("loop_sound", name)	Start ambient sound.
("repeat_sound", ...)	Repeat a sound with engine-supported arguments.
("stop_sound",)	Stop sound.
("start_playlist", ...)	Start a playlist; independent of drawing.

The "size" placeholder is interpreted in specific repeated/growing contexts, not as a universal numeric substitute.

5. Pattern and Sequence classes

Pattern(name="Pattern") supports .at(x, y), .shape(commands), .repeat(count), .grow(), .shrink(), .random_colors(palette), and .draw(). .draw() returns an instruction list. Coordinates passed to Pattern.at() must be integers. Pattern.repeat() requires a positive integer.

```
pattern = Pattern("Squares").at(-2, 0).shape(square_shape()).repeat(3).grow().draw()
```

Sequence() supports .add(name, event, sound=None) and .build(). Events may be a Pattern or a list of commands. In the fixed engine, each event starts with default colour black and width 3; unlike the original version, the fixed engine does not reset speed to 6 for each event.

The teacher engine also defines triangle_shape(), square_shape(), rectangle_shape(width=4, height=2), pentagon_shape(), hexagon_shape(), and star_shape(). These produce lower-level instruction lists for Pattern usage.

6. Animation, speed and controls

The fixed teacher engine accepts integer speed values from 0 to 100. Speed 0 means instant movement for straight segments and a single frame for a curve. Values 1–100 control pixels moved per animation frame; curves also use a speed-dependent angular limit. Screen updates are scheduled approximately every 20 ms, so perceived speed depends on rendering load.

On-screen controls: PAUSE/RESUME, RESTART, SPEED −, SPEED +, EXIT, HIDE/SHOW GRID, SOUND ON/OFF, HIDE/SHOW BUTTONS; coordinates and heading are displayed. Keyboard controls: Space (pause), H (buttons), G (grid), M (audio), Escape (exit).

Speed button changes affect the current engine speed. A subsequent shape that explicitly specifies speed=... can change it again. If a shape omits speed, it inherits the current setting.

7. Backgrounds and assets

The teacher engine accepts start(student_steps, student_colors, background=None). For a named background such as "beach", it searches a backgrounds folder alongside the engine and appends .png. Relative paths are joined to that backgrounds folder. PNG/GIF are handled by Tkinter; JPEG requires Pillow. "none" disables the background. Existing background asset names depend on the installed project folder.

```
background = "space"  # example setting in the student application
```

8. Examples and troubleshooting

Five stars in a row

```
stars = []
for i in range(5):
    stars += star(x=-4 + 2*i, y=0, size=1, color="purple", speed=20)
```

Filled rectangle

```
box = rectangle(x=-2, y=-1, width=4, height=2,
                color="blue", fill="yellow", line_width=5)
```

Curved S with thick line

```
my_s = letter_s(x=0, y=2, radius=1, color="purple", width=8, speed=10)
```

Common issues

Symptom	Check
SyntaxError on assignment inside image_sequence	Define variable before the list; reference it as ("Name", variable).
Unexpected keyword argument angle in letter_s	letter_s has no angle parameter; use raw curve commands.
Unexpected keyword argument width in rectangle	Use line_width for outline thickness.
No speed difference	Confirm updated engine and shape library are installed; check per-shape speed overrides.
Background not found	Check file name, backgrounds folder, and image format.
Shape off the grid	Adjust x/y, size, radius, or number/spacing of repeats.

9. Source/version notes

Source inspected: turtle_shapes_speed_fixed.py (student functions) and teacher_sequence_speed_fixed.py (engine). Some code examples from older PDF editions may describe earlier behaviour; this reference follows the inspected Python files. The complete application was not executed or integration-tested for this document.

10. Thicker lines (new)

Use the drawing instruction ("width", number) to set the pen thickness. A larger number produces a thicker stroke. Place the width instruction before the line or curve you want to draw.

```
my_line = [
   ("up", -3, 0),
   ("heading", 0),
   ("color", "blue"),
   ("width", 10),
   ("down",),
   ("forward", 6),
]

image_sequence = [
   ("Thick Blue Line", my_line),
]
```

Example widths: 2 = thin; 5 = medium; 10 = thick; 20 = extra thick. These are stroke-width values, not grid distances.

Shape-function parameters differ: for most shapes width= sets stroke thickness; rectangle() and venetian_blind() use line_width= for stroke thickness because width= controls the object's geometric size.

11. Easter egg logo gallery (project integration)

The later ASH Turtle project includes easter_eggs.py (drawing functions) and easter_egg_gallery.py (a ready-made list of named logo drawings). These are project-specific additions, not built-in Python Turtle functions.

Functions: nike_logo(), billabong_logo(), toyota_logo(), nasa_logo(), dominoes_logo(). They create simplified logo drawings rather than official vector artwork.

In student_images.py, keep the following import near the top and the extend call after the closing bracket of image_sequence. Both can remain commented out until the Easter eggs are needed:

```
# from easter_egg_gallery import easter_egg_sequence

image_sequence = [
   # ("My Drawing", my_drawing),
]

# image_sequence.extend(easter_egg_sequence)
```

To activate all five, remove # from both gallery lines. To activate only one, import the relevant logo function from easter_eggs and add a named pair inside image_sequence, e.g. ("Nike", nike_logo()), after importing it.

Important: do not call image_sequence.append(...) inside the image_sequence = [...] list literal. Either put ("Name", commands) pairs inside the list, or append/extend only after its closing].

12. Version and verification note

Sections 10–11 document the later app workflow and thick-line instruction. Earlier sections retain their original source/version scope. Behaviour should be checked in the specific app version being used, particularly fill and curve rendering.